



实验二：深度学习正则化 技术实践

《深度学习》实验课
2026年 春

目录

1	PyTorch深度学习通用流程
2	过拟合与正则化
3	深度学习中常见正则化技术
4	实验作业

PyTorch深度学习通用流程

➤ 回顾实验一给出的MNIST数据案例如下图所示：

PyTorch搭建神经网络：

- 第一步：导入相关模块，如`import torch`。
- 第二步：设置设备，检查是否有可用的GPU，如果有则使用GPU，否则使用CPU
- 第三步：数据加载和预处理，如使用`torchvision.datasets.MNIST`加载MNIST数据集，使用`transforms`进行数据预处理和标准化。
- 第四步：逐层搭建网络结构，如 `model = nn.Sequential()`。
- 第五步：配置训练方法，包含优化器、损失函数。
- 第六步：执行训练循环过程，如使用 `model.train()` 设置模型为训练模式。
- 第七步：使用`model.eval()`做模型评估，返回测试集上的损失和准确率。

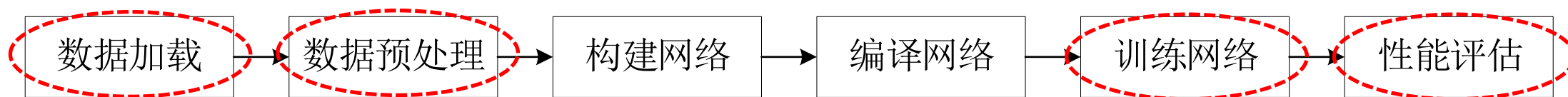
在GPU环境的运行结果如下：

```
使用设备: cuda
Epoch 1 completed
Epoch 2 completed
Epoch 3 completed
Epoch 4 completed
Epoch 5 completed
Test set: Average loss: 0.0788, Accuracy: 9752/10000 (97.52%)
```

在CPU环境的运行结果如下：

```
使用设备: cpu
Epoch 1 completed
Epoch 2 completed
Epoch 3 completed
Epoch 4 completed
Epoch 5 completed
Test set: Average loss: 0.0801, Accuracy: 9752/10000 (97.52%)
```

➤ 总结 Pytorch 深度学习的通用流程如图所示，分为以下6个步骤。



PyTorch深度学习通用流程

深度学习的通用流程分为以下6个步骤。

- **数据加载**，加载用于训练深度神经网络的数据，使得深度神经网络能够学习到数据中潜在的特征，可以指定从某些特定路径加载数据。
- **数据预处理**，对加载的数据进行预处理，使之符合网络的输入要求，如**标签格式转换**、**样本变换**等。数据形式的不统一将对模型效果造成较大的影响。
- **构建网络**，根据特定的任务使用不同的网络层**搭建网络**，若网络太简单则无法学习到足够丰富的特征，若网络太复杂则容易过拟合。
- **编译网络**，设置网络训练过程中使用的**优化器和损失函数**，优化器和损失函数的选择会影响到网络的训练时长、性能等。
- **训练网络**，通过不断的迭代和**批训练**的方法，调整模型中**各网络层的参数**，减小模型的损失，使得模型的预测值逼近真实值。
- **性能评估**，计算网络训练过程中**损失和分类精度**等与模型对应的评价指标，根据评价指标的变化调整模型从而取得更好的效果。

PyTorch深度学习通用流程——补充知识点



1、数据加载

- 在PyTorch框架中，`torchtext.utils`包中的类可以用于数据加载。
- 常见加载的数据的方式有从指定的网址（`url`）下载数据、加载本地的表格数据和直接读取压缩文件中的数据。

(1) 从指定的网址下载文件 `torchtext.utils.download_from_url(url, path=None, root='.data', overwrite=False, hash_value=None, hash_type='sha256')`

(2) csv文件读取器 `torchtext.utils.unicode_csv_reader(unicode_csv_data, **kwargs)`

(3) 读取压缩文件数据 `torchtext.utils.extract_archive(from_path, to_path=None, overwrite=False)`

PyTorch深度学习通用流程——补充知识点

2、数据预处理

- 在深度学习的时候，可以事先单独将图片进行清晰度、画质和切割等处理然后存起来以扩充样本，但是这样做效率比较低下，而且不是实时的。
- PyTorch对数据进行处理分为**图像数据预处理**和**文本数据预处理**，在PyTorch框架中分别用**torchvision库**、**trochtext库**（本次实验主要介绍torchvision库内容）。
- 在PyTorch框架中，处理图像与视频的torchvision库中常用的包及其说明如表所示。

包	说明
torchvision.datasets	提供数据集下载和加载功能，包含若干个常用数据集
torchvision.io	提供执行IO操作的功能，主要用于读取和写入视频及图像
torchvision.models	包含用于解决不同任务的网络结构，并提供已预训练过的模型，包括图像分类、像素语义分割、对象检测、实例分割、人物关键点检测和视频分类
torchvision.ops	主要实现特定用于计算机视觉的运算符
torchvision.transforms	包含多种常见的图像预处理操作，如随机切割、旋转、数据类型转换、图像到tensor、numpy数组到tensor、tensor到图像等
torchvision.utils	用于将形似(3×H×W)的张量保存到硬盘中，能够制作图像网络



2、数据预处理（图像/视频）

其中torchvision.transforms中常用的4种图像预处理操作如下。

（1）组合图像的多种变换处理

- Compose类可以将多种图像的变换处理组合到一起，Compose类语法格式如下，其中参数“transforms”接收的是由多种变换处理组合成的列表。

`torchvision.transforms.Compose(transforms)`

（2）对图像做变换处理

- 常见的PIL图像包括：L（灰色图像）、P（8位彩色图像）、I（32位整型灰色图像）、F（32位浮点灰色图像）、RGB（8位彩色图像）、YCbCr（24位彩色图像）、RGBA（32位彩色模式）、CMYK（32位彩色图像）、1（二值图像）等模式。

PyTorch深度学习通用流程——补充知识点



2、数据预处理（图像/视频）

在PyTorch框架下能实现对PIL图像做变换处理的类较多，此处介绍常用的6个类。

- CenterCrop类，可以裁剪给定的图像并返回图像的中心部分，语法格式如下，其中参数“size”指的是图像的期望输出大小。

`torchvision.transforms.CenterCrop(size)`

- ColorJitter类，随机改变图像的亮度、对比度、饱和度和色调，语法格式如下。

`torchvision.transforms.ColorJitter(brightness=0, contrast=0, saturation=0, hue=0)`

- FiveCrop类可以将图像裁剪成四个角和中心部分
- Grayscale类可以将图像转换为灰度图像。
- Pad类可以使用给定的填充值填充图像的边缘区域。
- Resize类可以将输入图像调整到指定的尺寸，语法格式如下。

`torchvision.transforms.Resize(size, interpolation=<InterpolationMode.BILINEAR: 'bilinear'>)`



2、数据预处理（图像/视频）

（3）对图像数据做变换处理

对图像数据做变换处理操作的类较多，此处介绍常用的两个类。

➤ **LinearTransformation**类，用平方变换矩阵和离线计算的均值向量变换图像数据，语法格式如下：

```
torchvision.transforms.LinearTransformation(transformation_matrix, mean_vector)
```

参数 *transformation_matrix* 表示变换矩阵； *mean_vector* 表示平均向量。

➤ **Normalize**类，用均值和标准差对图像数据进行归一化处理，语法格式如下：

```
torchvision.transforms.Normalize(mean, std, inplace=False)
```

参数 *mean* 表示每个通道的均值； *std* 表示每个通道的标准差。

PyTorch深度学习通用流程——补充知识点



2、数据预处理——图像/视频

(4) 格式转换变换处理

由于直接读取图像得到的数据的类型与输入网络的数据类型不对应，因此需要对数据的格式进行转换。

➤ ToPILImage类

- ToPILImage类将tensor类型或ndarray类型的数据转换为PIL Image类型的数据。
- ToPILImage类的语法格式如下，其中参数“mode”指的是输入数据的颜色空间和像素深度。

`torchvision.transforms.ToPILImage(mode=None)`

➤ ToTensor类

- ToTensor类将PIL图像或numpy数组转换为PyTorch张量（tensor）。
- 将图像像素值从0-255的整数范围转换到0-1的浮点数范围（即除以255）。
- ToTensor类的语法格式如下。

`torchvision.transforms.ToTensor()`

备注：对于MNIST数据集，图像是灰度图像，所以每个像素只有一个通道。使用`transforms.ToTensor()`会将一个尺寸为（H, W）的PIL图像或numpy数组转换为一个尺寸为（C, H, W）的张量，其中C是通道数（对于灰度图C=1）。

PyTorch深度学习通用流程——补充知识点



3、构建网络

主要是使用Sequential容器构建网络，多种激活函数搭配使用，具体内容可以参考实验一资料，后面实验四也会对各类常用神经网络结构做具体实践应用。

4、编译网络

主要是选择合适的损失函数和优化器（参考实验一资料），另外实验三也会单独做优化器实践。

PyTorch深度学习通用流程——补充知识点



5、训练网络——迭代次数epoch

- 训练网络时，通过设置epoch参数调整训练中网络的迭代次数，epoch的值为多少就表示所有训练样本被传入网络训练多少轮（如epoch为1，表示所有训练样本被传入网络训练一轮）。
- 当迭代次数的值设置太小时，训练得到的模型效果较差，可以适当增加迭代次数的大小，从而优化模型的效果。但当迭代次数的值太大时（数据特征有限），可能会导致模型过拟合、训练时间过长等问题，用户可以根据损失值和准确率的变化趋势适当调整迭代次数的值，当趋势逐渐平稳时，迭代次数的值设得较为理想。

```
# 训练模型
model.train() # 将模型设置为训练模式，这会启用dropout和batch normalization等训练特定行为
for epoch in range(5): # 训练5个epoch
    for data, target in train_loader: # 遍历训练数据加载器中的每个批次
        data, target = data.to(device), target.to(device) # 将数据移动到指定设备

        optimizer.zero_grad() # 清除之前的梯度，防止梯度累积
        output = model(data) # 前向传播，计算模型输出
        loss = criterion(output, target) # 计算损失
        loss.backward() # 反向传播，计算梯度
        optimizer.step() # 更新模型参数

    print(f'Epoch {epoch+1} completed') # 打印当前epoch完成信息
```

PyTorch深度学习通用流程——补充知识点



5、训练网络——批训练batch_size

- 为了更好地利用**GPU**的计算能力，一般在网络的训练过程中会同时计算多个样本，这种训练方式被称为**批训练**。运用批训练的好处主要有以下几种。
 - 内存利用率提高，大矩阵乘法的并行化效率提高。
 - 跑完一次数据集所需的迭代次数减少，对相同数据量的处理速度进一步加快。
 - 通常在合理的范围内，训练中使用的批量大小越大，其确定的下降方向越准确，引起训练的震荡越小。
- 同样用户可以根据自己的需求设置批量大小参数的大小，一般根据用户的GPU显存资源来设置，若设置的值太大，当显存不足时，可能会导致训练终止；若设置的值太小，没有充分利用GPU计算资源，使得训练速度较慢，所以用户可以适量减少batch_size值来减少训练的显存使用量。

```
# 数据预处理和加载
# transforms.Compose 将多个数据转换操作组合在一起
transform = transforms.Compose([
    transforms.ToTensor(), # 将PIL图像或numpy数组转换为PyTorch张量
    transforms.Normalize((0.1307,), (0.3081,)) # 使用MNIST数据集的均值和标准差进行标准化
    # 标准化公式: (input - mean) / std
])

# 加载MNIST数据集
# datasets.MNIST是PyTorch提供的MNIST数据集加载器
# root: 数据集存储路径
# train=True: 加载训练集
# download=True: 如果数据集不存在则自动下载
# transform: 应用上面定义的数据转换
train_dataset = datasets.MNIST('./data', train=True, download=True, transform=transform)
test_dataset = datasets.MNIST('./data', train=False, download=True, transform=transform)

# 创建数据加载器
# DataLoader负责批量加载数据，支持多进程数据加载和数据打乱
# batch_size=64: 每个批次包含64个样本
# shuffle=True: 在每个epoch开始时打乱训练数据顺序
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False) # 测试集不需要打乱
```

6、性能评估

- 神经网络的实际应用大致可以分成回归问题和分类问题。
- 在回归问题中，常用的评估指标有平均绝对值误差、均方误差、均方根误差等。其计算方式与损失的计算方式大致相同。
- 在分类问题中，常用的评估指标有混淆矩阵（也称误差矩阵，Confusion Matrix）、ROC曲线、AUC面积3种。
- 其中，混淆矩阵是绘制ROC曲线的基础，同时它也是衡量分类模型准确度中最基本、最直观、计算过程常用方法之一。
- 一个简单的二分类问题的混淆矩阵如下图所示。

		实际结果	
		正例	负例
预测结果	正例	TP	FN
	负例	FP	TN

- 混淆矩阵中TP、TN、FP和FN的含义如下。
 - TP (True Positives)：正确地将正例预测为正例的分类数。
 - TN (True Negatives)：正确地将负例预测为负例的分类数。
 - FP (False Positives)：错误地将负例预测为正例的分类数。
 - FN (False Negatives)：错误地将正例预测为负例的分类数。



6、性能评估

- 由于混淆矩阵中统计的是个数，在面对大量的数据，光凭算个数，很难衡量模型的优劣，因此在混淆矩阵的统计结果上提出了4个指标：准确率、精确度、召回率和F1值。

- **准确率**（Accuracy）为预测正确的结果占总样本的百分比，计算公式如下所示。

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \times 100\%$$

- **精确度**（Precision）是指在一定实验条件下多次测定的平均值与真实值相符合的程度，以误差来表示，用于表示系统误差的大小，计算公式如下。

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \times 100\%$$

- **召回率**（Recall）是广泛用于信息检索和统计学分类领域的度量值，用于评价结果的质量，计算公式如下所示。

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \times 100\%$$

- **F-Measure**又称为F-Score，综合考虑精确度与召回率，计算公式如下。

$$\text{F-Measure} = \frac{2 * \text{Recall} * \text{Precision}}{\text{Recall} + \text{Precision}}$$

PyTorch深度学习通用流程——补充知识点



6、性能评估

```
# 模型评估
model.eval() # 将模型设置为评估模式，这会禁用dropout和batch normalization等训练特定行为
test_loss = 0 # 初始化测试损失
correct = 0 # 初始化正确预测的样本数
total = 0 # 初始化总样本数

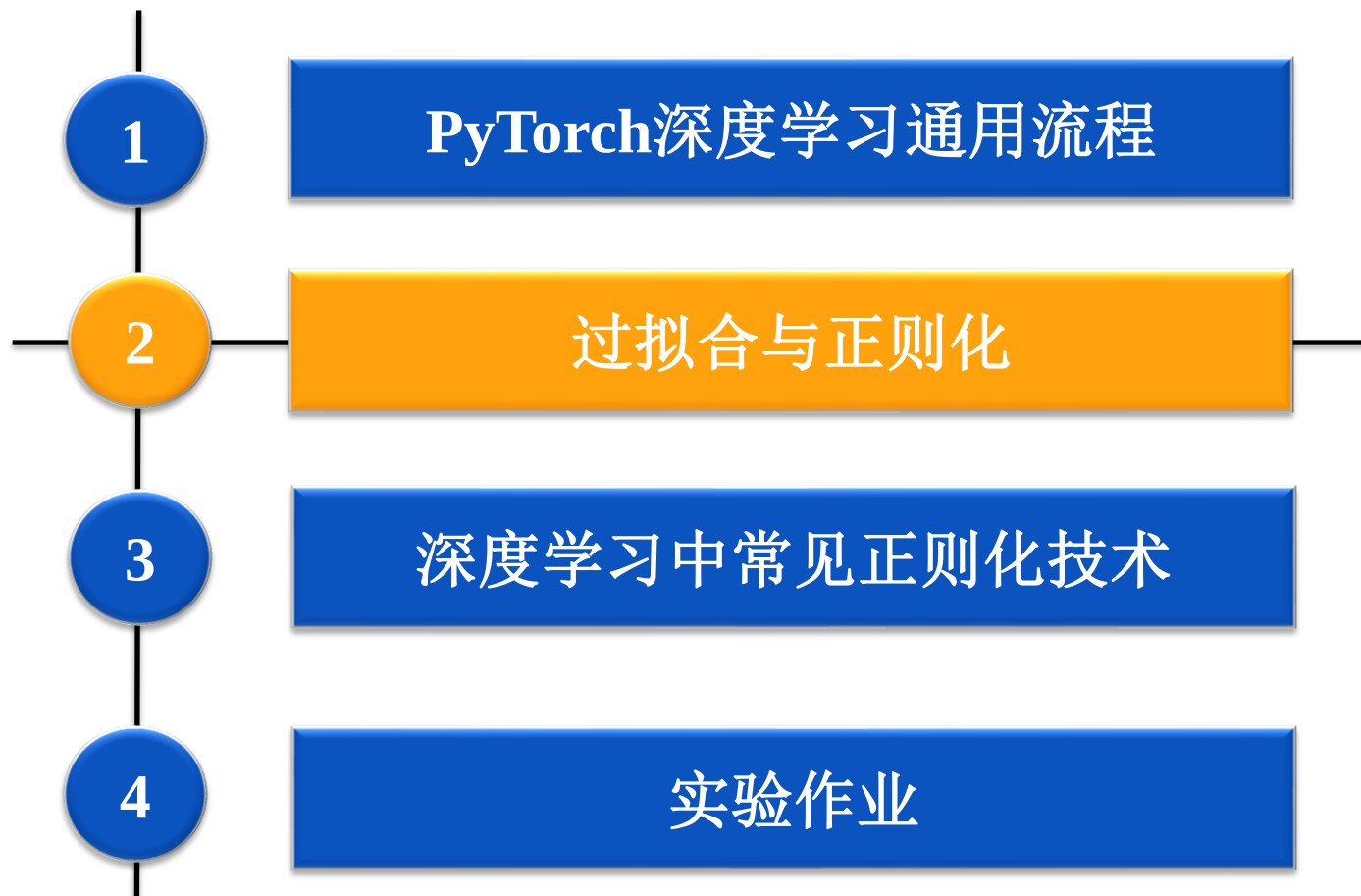
with torch.no_grad(): # 禁用梯度计算，节省内存和计算资源
    for data, target in test_loader: # 遍历测试数据加载器中的每个批次
        data, target = data.to(device), target.to(device) # 将数据移动到指定设备
        output = model(data) # 前向传播，计算模型输出
        test_loss += criterion(output, target).item() # 累加批次损失
        pred = output.argmax(dim=1, keepdim=True) # 获取预测结果（概率最大的类别）
        correct += pred.eq(target.view_as(pred)).sum().item() # 统计正确预测的数量
        total += target.size(0) # 累加样本数量

# 计算平均测试损失和准确率
test_loss /= len(test_loader) # 平均损失 = 总损失 / 批次数量
accuracy = 100. * correct / total # 准确率 = 正确预测数 / 总样本数 * 100%

# 打印测试结果
print(f'Test set: Average loss: {test_loss:.4f}, Accuracy: {correct}/{total} ({accuracy:.2f}%)')
```

在实践中常用准确率来做性能评估。

目录



过拟合

➤ 过拟合:

- 随着模型复杂度增加，训练集上的损失不断降低，但测试损失却是先降后增，如下图1所示，即为过拟合。
- 造成过拟合的原因：
 - ① 是数据过少且训练过度，小批量数据的特点无法代表总体;
 - ② 是模型复杂度高，拟合能力过强，陷入了局部最优。比如一条直线就可以区分的，模型拟合成凹凸凸凸的曲线，如下图4所示。

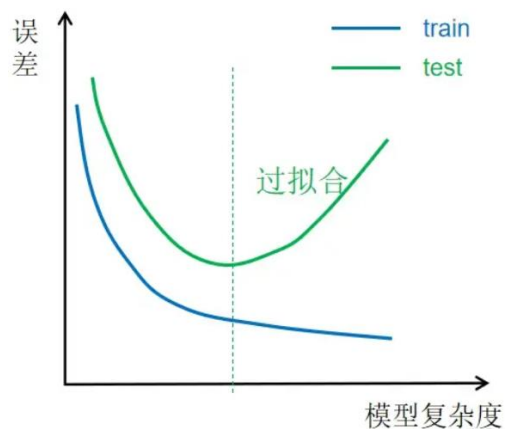


图1 过拟合现象

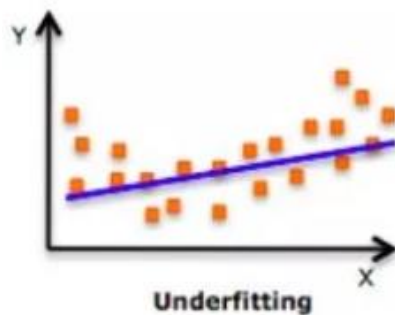


图2 欠拟合



图3 正拟合

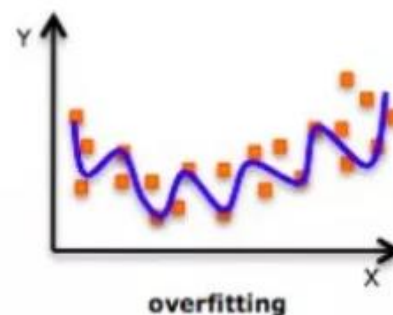


图4 过拟合

正则化如何减少过拟合

➤ 过拟合:

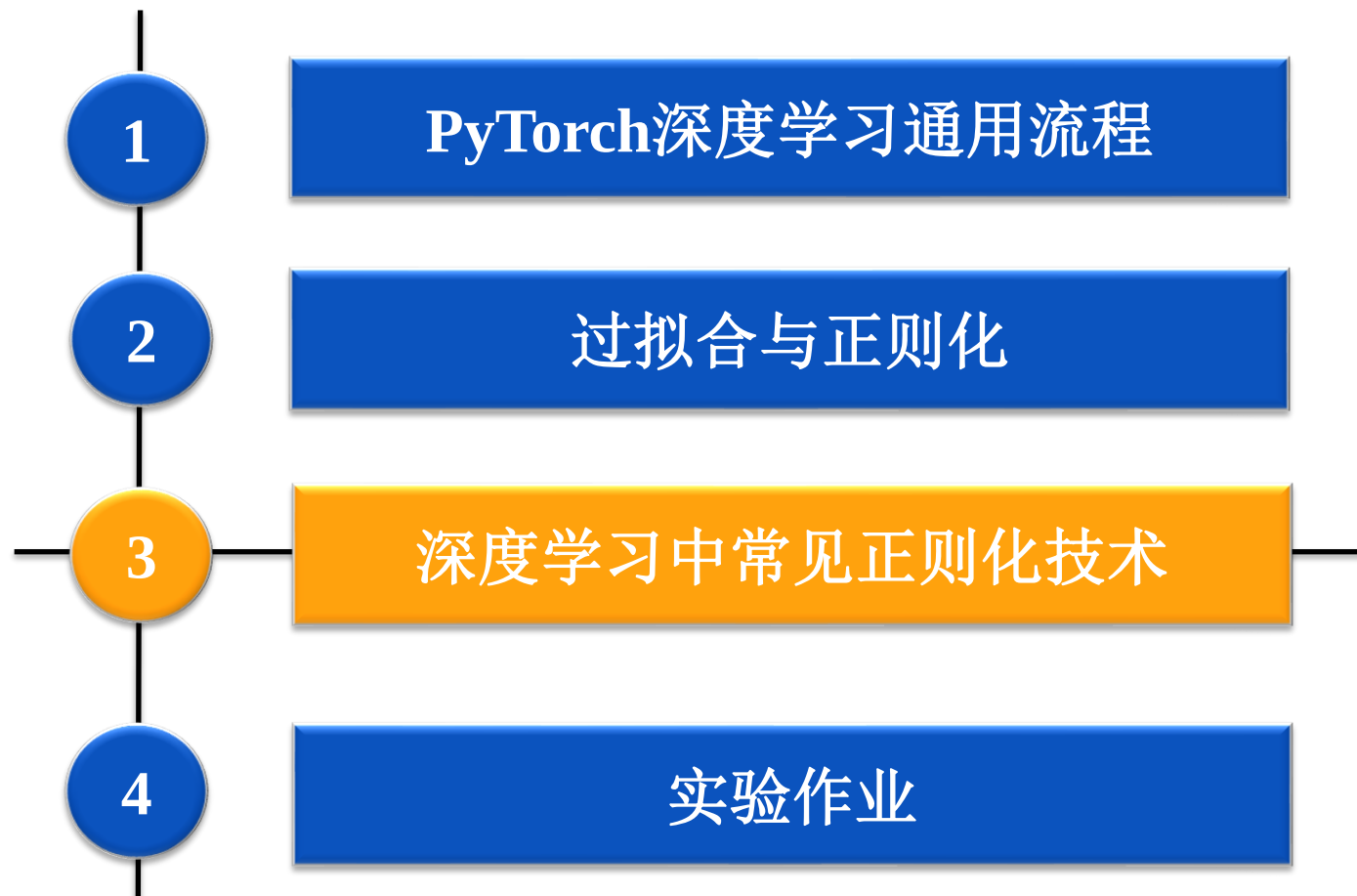
• 解决过拟合的方法:

- ① 可以扩充样本量做训练，提升模型的泛化能力。
- ② 在没有扩充样本量的可能下，只能降低模型的复杂度。两条路径：一是进行降维，进行特征约减，这样可以减少模型参数的个数；二是对参数进行约束，在模型的优化目标（损失）中加入对参数规模的惩罚项，使得参数的取值范围减少（该方法也称正则化）。

➤ 正则化

- 当参数越多或取值越大时，该惩罚项就越大。通过调整惩罚项的权重系数，可以使模型在“尽量减少训练损失”和“保持模型的泛化能力”之间取得平衡。通过调整正则化项的权重，可以灵活设置模型参数数量和训练集上的损失，从而限制模型能力，避免过拟合。

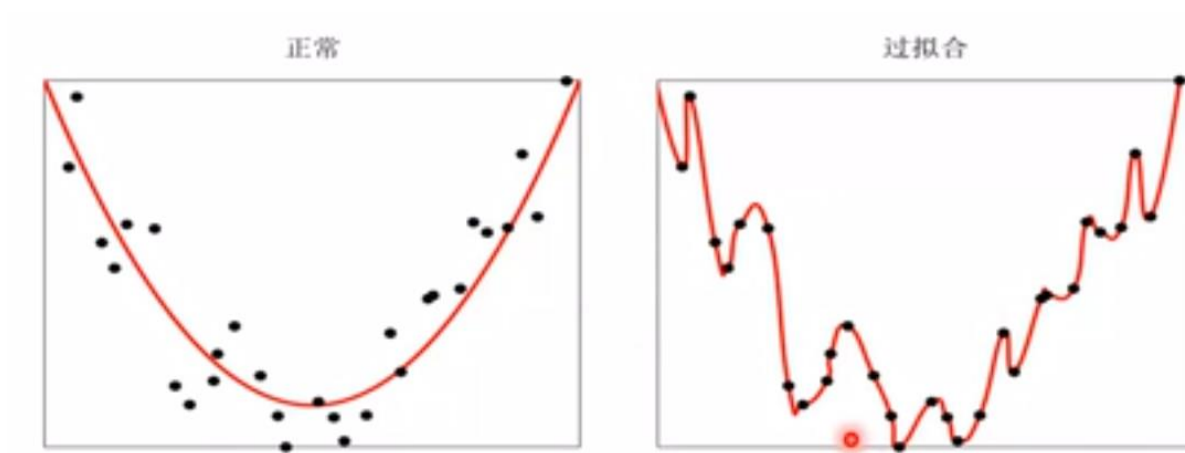
目录



深度学习中常见正则化技术

常见正则化技术有：

- L1和L2正则化
- Dropout
- 提前停止
- 数据增强



深度学习中常见正则化技术

➤ 1、L1和L2正则化

- L1和L2是最常见的正则化方法。它们在损失函数中增加一个正则项，使得权重矩阵的值减小，从而模型简单化，在一定程度上能减少过拟合。但这个正则化项在L1和L2中是不同的。

- L2正则化
$$J(\omega, b) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \|\omega\|_2^2$$

其中 λ 是正则化参数，它是一个需要优化的超参数。L2正则化又称为权重衰减，因为其导致权重趋向于0（但不全是0）

- L1正则化
$$J(\omega, b) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \|\omega\|_1$$

其中 λ 为正则化参数，是超参数，不同于L2，权重值可能会被减少到0。

• L1和L2正则化区别

- ① L2让所有特征的系数都缩小，但是不会减为0，它会使优化求解稳定快速，所以L2适用于特征之间没有关联的情况；
- ② L1可以让一部分特征的系数缩小到0，从而间接实现特征选择，所以L1适用于特征之间有关联的情况。

深度学习中常见正则化技术

➤ 1、L1和L2正则化

- 以MNIST数据集为例，L2正则化直接在优化器中通过`weight_decay`参数实现，这是更高效的方式。示例代码如下所示：

```
optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=0.001)
```

注意：这里 `lr` 为优化器的学习率参数，`weight_decay` 是正则化参数的值，它需要被进一步优化，可以调试的。

- L1正则化代码，需要手动计算模型参数的L1范数并添加到损失函数中，如下：

```
l1_lambda = 0.001 # L1正则化系数
```

```
# 训练模型
model.train()
for epoch in range(5):
    for data, target in train_loader:
        data, target = data.to(device), target.to(device)

        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, target)

        # 添加L1正则化项
        l1_reg = torch.tensor(0.).to(device)
        for param in model.parameters():
            l1_reg += torch.norm(param, 1)
        loss += l1_lambda * l1_reg

    loss.backward()
    optimizer.step()

print(f'Epoch {epoch+1} completed')
```

深度学习中常见正则化技术

➤ 2、Dropout

- 假设我们的神经网络结构如下图1所示。
- Dropout工作内容：每次迭代，随机选择一些节点，将它们连同相应的输入和输出一起删掉，如图2所示。所以，每一轮迭代都有不同的节点集合，这也导致了不同的输出。
- 选择丢弃多少节点的概率是dropout函数的超参数，如图3所示，dropout可以被用在隐藏层以及输入层。

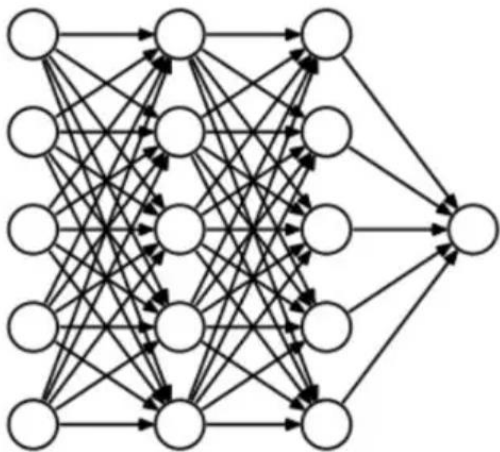


图1

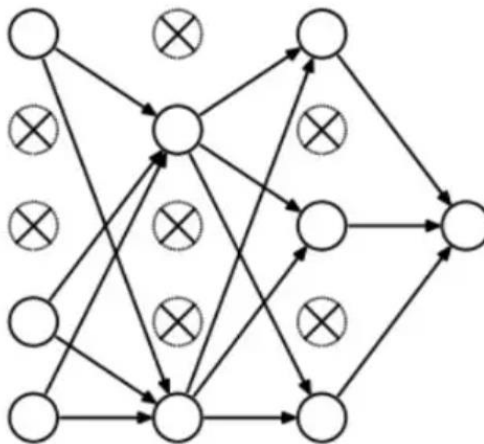


图2

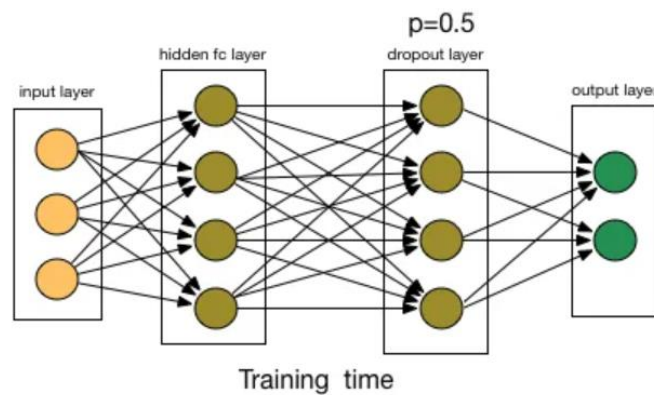


图3

深度学习中常见正则化技术

➤ 2、Dropout

- 当我们有较大的神经网络时，为了引入更多的随机性，通常会优先使用dropout。其实现方法，用于每一个隐藏层中，语法格式如下：

`torch.nn.Dropout(p=0.5, inplace=False)`

- ☐ 参数**p**: float类型，表示张量元素被置0的概率，默认为0.5;
- ☐ 参数**inplace**: int类型，表示是否原地执行，默认为False。
- 下面是对应的Python代码，这里定义每个神经元的丢弃概率为0.5，一般设置在0.1-0.5之间。

```
# 建立全连接神经网络模型
model = nn.Sequential(
    nn.Flatten(), # 将 28x28 图像展平为 784 维向量
    nn.Linear(784, 128), # 全连接层, 128 个神经元
    nn.ReLU(), # ReLU 激活函数
    nn.Linear(128, 10), # 输出层, 10 个神经元
    nn.Dropout(p=0.5) # 设置dropout=0.5
).to(device)
```

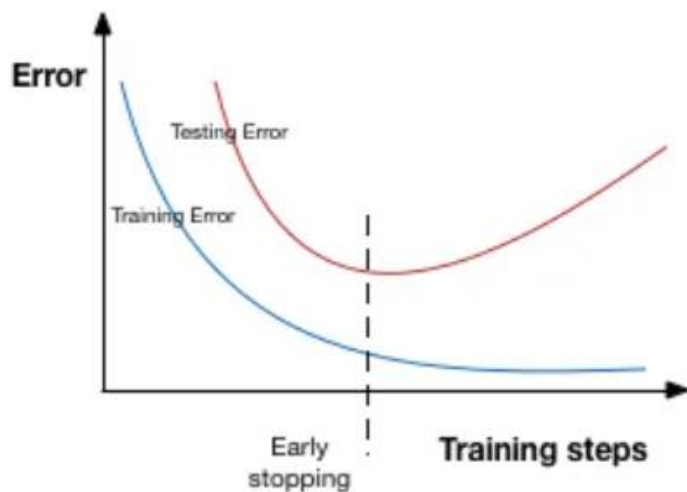
调整参数的原则:

- ☐ 过拟合严重时 → 提高Dropout率
- ☐ 欠拟合时 → 降低Dropout率
- ☐ 尝试增加网络层数或每层的神经元数量，观察Dropout的效果
- ☐ 尝试不同的Dropout位置（如在全连接层/卷基层等等）

深度学习中常见正则化技术

➤ 3、提前停止(Early stopping)

- 提前停止是一种交叉验证的策略，即把一部分**训练集**保留作为验证集。当看到验证集上的性能变差时，就立即停止模型的训练。
- 在下图中，我们在**虚线处**停止模型的训练，因为在此处之后模型会开始在训练数据上过拟合。



- 以MNIST数据集处理为例，下面是示例代码，
(1) 划分训练集、验证集以及测试集：

```
# 数据加载
train_data = datasets.MNIST('./data', train=True, download=True, transform=transform)
test_data = datasets.MNIST('./data', train=False, transform=transform)
train_data, val_data = random_split(train_data, [48000, 12000])

train_loader = DataLoader(train_data, batch_size=64, shuffle=True)
val_loader = DataLoader(val_data, batch_size=64)
test_loader = DataLoader(test_data, batch_size=64)
```

深度学习中常见正则化技术

➤ 3、提前停止(Early stopping)

(2) 开启训练和验证

```
# 训练与早停
best_loss = float('inf')
patience, counter = 5, 0

for epoch in range(50):
    # 训练
    model.train()
    for x, y in train_loader:
        x, y = x.to(device), y.to(device)
        optimizer.zero_grad()
        loss = criterion(model(x), y)
        loss.backward()
        optimizer.step()

    # 验证
    model.eval()
    val_loss = 0
    with torch.no_grad():
        for x, y in val_loader:
            x, y = x.to(device), y.to(device)
            val_loss += criterion(model(x), y).item()
    val_loss /= len(val_loader)
```

(3) 做提前停止的检查判断，并保存最优模型

```
# 早停检查
if val_loss < best_loss:
    best_loss = val_loss
    counter = 0
    torch.save(model.state_dict(), 'best_model.pth')
else:
    counter += 1
    if counter >= patience:
        print(f"早停于第 {epoch+1} 轮")
        break

print(f"轮次 {epoch+1}: 验证损失 = {val_loss:.4f}")
```

(4) 加载最优模型，做测试

```
# 加载最佳模型并测试
model.load_state_dict(torch.load('best_model.pth'))
model.eval()
correct = 0
with torch.no_grad():
    for x, y in test_loader:
        x, y = x.to(device), y.to(device)
        pred = model(x).argmax(dim=1)
        correct += (pred == y).sum().item()

print(f"测试准确率: {100 * correct / len(test_data):.2f}%")
```

深度学习中常见正则化技术

➤ 4、数据增强

- 图像增强的增强主要是通过算法对图像进行转变，引入噪声等方法来增加数据的多样性以及训练数据量。
- 图像增强的增强方法有：

(1) 随机旋转 (RandomRotation) `transforms.RandomRotation(10)`

将图像随机旋转 ± 10 度，使模型对数字的方向变化更鲁棒。

(2) 随机仿射变换 (RandomAffine) `transforms.RandomAffine(0, translate=(0.1, 0.1))`

在水平和垂直方向上随机平移最多10%，模拟手写数字的位置变化。

(3) 随机缩放裁剪 (RandomResizedCrop) `transforms.RandomResizedCrop(28, scale=(0.9, 1.1))`

随机缩放图像并在28x28大小裁剪，增加对数字大小的鲁棒性。

.....

其他更多的数据增强的方法可参考前面PPT中P6-10页数据预处理的内容，MNIST数据集具体的代码示例如右图所示：

```
# 数据增强
train_tf = transforms.Compose([
    transforms.RandomRotation(10),
    transforms.RandomAffine(0, translate=(0.1, 0.1)),
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

test_tf = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

# 数据加载
train_data = datasets.MNIST('./data', train=True, download=True, transform=train_tf)
test_data = datasets.MNIST('./data', train=False, transform=test_tf)
```

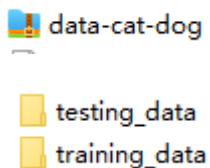
目录



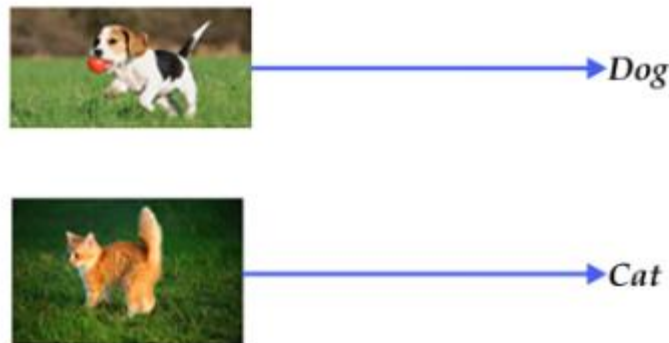
实验作业——内容

➤ 基于**猫狗数据集**，构建多层的神经网络结构，实现模型训练与预测，并得出相应结论。要求：

- 1、请结合目前所学的神经网络内容，**构造三种不同复杂度的神经网络结构**；
- 2、结合前面介绍的**torchvision库**，实现**数据集的数据加载与预处理**；
- 3、**实践四种正则化方法**，调试超参数，得到最优参数，来减少模型过拟合；
- 4、需要调试学习率、批处理、迭代次数等参数，**记录调参过程以及绘制损失函数和准确率的变化趋势图**。
- 5、完成相应的**实验报告**。



猫狗数据集，其中**training_data**训练数据有25000张图片，**testing_data**测试数据有400张，猫狗图片各占二分之一。



实验作业——内容

➤ 具体细节要求:



1、构造三种不同复杂度的神经网络结构:

- 简单网络: 2-3个隐藏层, 每层神经元数不超过128
- 中等网络: 4-5个隐藏层, 包含Dropout层
- 复杂网络: 6+个隐藏层, 结合多种正则化技术

2、数据加载预处理:

- 使用torchvision完成数据加载和预处理
- 实现数据增强(至少3种变换)
- 创建训练集、验证集、测试集
- 使用DataLoader实现批量加载

3、实践四种正则化方法:

- L1/L2正则化、Dropout、提前停止、数据增强

4、训练预测、调参绘图:

- 实现自定义的训练循环
- 调试学习率、批大小等超参数, 记录训练过程中的指标变化
- 模型保存和加载功能
- 使用matplotlib或seaborn绘制损失和准确率曲线
- 在GPU环境中运行(如可用)

5、实验报告

- 记录每次实验的超参数设置和结果
- 分析不同正则化方法的效果
- 比较三种网络结构的优缺点
- 总结过拟合问题的解决方案

实验作业——提交方式

实验报告提交至平台 <https://grader.hitsz.edu.cn/courses>

注意：

- 1、用户名用**统一身份登录**；
- 2、请提交到相应的条目「2026春-深度学习」课程 - 实验二；
- 3、提交截止时间：**下周二 5月26号 晚24点**；
- 4、文件夹&压缩包命名要求：**学号_姓名_深度学习实验二**
- 5、提交内容：实验报告**(.pdf文件)**+代码**(.py/ipynb文件)**，重要代码处加注释)，一起打包为**zip格式**压缩包，实验报告参考模板。